High-Performance Order Matching Engine — Project Overview

> ***46,500+ lines of C++20*** *| 84 headers | 14 source files | 77 test files | 426 CTest targets*
>
> *A C++20 low-latency matching engine drawing on institutional exchange design principles —* ***237 ns P50 core-matching latency (Clang PGO) validated on x86 Xeon bare metal*** *(≈125 ns on Apple Silicon) — with horizontal scalability.*

---

1. Executive Summary

This is a C++20 low-latency order matching engine with institutional-grade architecture drawing on exchange design principles. It implements O(1) price-level lookup via `FlatPriceMap`, lock-free MPSC queues, thread-per-symbol horizontal scaling, CRC-32 journaling with deterministic replay, four wire protocols (FIX 4.2/4.4, OUCH 4.2, ITCH 5.0, SBE) over both real TCP and UDP transports, MoldUDP64 multicast with gap-recovery retransmission service, TLA+-verified safety invariants, cross-host log replication, and a complete operational stack including config management, webhook alerting, Prometheus metrics, and Docker deployment.

Codebase Statistics

Metric	Count
Total C++ LOC	~46,500
Header files (`include/`)	84
Source files (`src/`)	14
Test files	77
Individual test cases	426 CTest targets
TLA+ specifications	12 (1.26M states — matching + book layers)
Documentation files	6 (plus architecture/benchmark docs)

---

2. Core Engine Architecture

System Topology

```
          ORDER ENTRY                                 MARKET DATA
          ───────────                                 ───────────

  ┌──────────────┐    ┌──────────────┐         ┌──────────────────┐
  │ FIX 4.2/4.4  │    │   OUCH 4.2   │         │    ITCH 5.0      │
  │    (text)    │    │   (binary)   │         │    (binary)      │
  └──────┬───────┘    └──────┬───────┘         └────────▲─────────┘
         │                   │                          │
         │                   │                          │
                      ┌──────▼──────┐
```

```
         │            │ SoupBinTCP │             ┌─────┴──────┐
         │            │  session   │             │ MoldUDP64  │
         │            └─────┬──────┘             │ multicast  │
         │                  │                    └─────▲──────┘
   ┌─────▼──────┐    ┌──────▼─────┐    ┌────────────┐  │
   │ FixSession │    │ OuchSession│    │ SbeSession │  │ ItchPublisher
   └─────┬──────┘    └──────┬─────┘    └─────┬──────┘  │  (per book)
         │                  │                │         │
         └──────────────────┼────────────────┘         │
                            ▼                          │
                  ┌───────────────────┐                │
   Admin :8080 ──▶│   MatchingEngine  │────events──────┘
                  │   (order router)  │
                  └─┬───┬───┬───┬─────┘
                    │   │   │   │
                ┌───▼─┐ │ ┌─▼───▼──┐
                │ Q[0]│ │ │ Q[1..N]│   MpscQueue per thread
                └──┬──┘ │ └──┬─────┘
                   │    │    │
              ┌────▼──┐ │ ┌──▼────┐
              │Thread0│ │ │Thread1│    Worker threads
              │ BTC,  │ │ │ SOL,  │    (symbol affinity)
              │ ETH   │ │ │ AVAX  │
              └───────┘ │ └───────┘
                        │
                 ┌──────▼──────┐    ┌──────────────┐    ┌─────────────────┐
                 │  OrderBook  │───▶│ MoldPacket   │◀───│ ItchRetransmit  │
                 │  ObjectPool │    │   Journal    │    │  Service (TCP)  │
                 │  Journal    │    └──────────────┘    └─────────────────┘
                 └─────────────┘     gap recovery        re-request replay
```

Order Types Supported

- **Standard**: Limit, Market
- **Time-In-Force**: IOC (Immediate-or-Cancel), FOK (Fill-or-Kill), DAY, GTD (Good-Til-Date)
- **Conditional**: Stop, StopLimit, TrailingStop
- **Institutional**: Pegged, Iceberg (hidden quantity), Hidden, PostOnly, MIT, MOC, LOC
- **Match Algorithms**: Price-Time FIFO and Pro-Rata allocation with LMM/DMM floor guarantee (40% of available qty) and rounding-remainder priority

Core Data Structures

Component	Purpose	Complexity
`FlatPriceMap`	Price-level lookup by tick index	O(1) insert/lookup
`FlatHashMap`	Robin-Hood open-addressing hash map	O(1) amortized
`IntrusiveList`	Doubly-linked order queue per price level	O(1) insert/remove
`ObjectPool`	Pre-allocated slab allocator for `Order` nodes	O(1) alloc/dealloc
`RingBuffer`	Cache-line aligned lock-free ring buffer	O(1) push/pop

`MpscQueue`	Multi-producer, single-consumer lock-free queue	O(1) push/pop

---

3. Concurrency & Networking

Thread-Per-Symbol Partitioning

N worker threads with independent lock-free `MpscQueue` ring buffers. Orders are deterministically routed by `hash(symbolId) % numThreads`, eliminating cross-thread contention on the hot path.

Multi-Protocol Order Entry

All four protocols dispatch into the same `MatchingEngine`.

- **FIX 4.2 / 4.4**: ASCII text + checksum, full session-layer state machine. `TransactTime` validation and version negotiation per accepted `BeginString`.
- **OUCH 4.2**: NASDAQ binary order entry, big-endian fixed-width.
- **SBE (Simple Binary Encoding)**: FIX-TG schema-driven binary (CME / ICE style). Forward and backward compatibility. Encode speed of ~1 ns/op.
- **SoupBinTCP**: TCP envelope wrapping OUCH on the wire.

Market Data Stack

- **ITCH 5.0 Publisher**: Converts engine events to ITCH 5.0 frames.
- **MoldUDP64 Multicast**: Batched publisher with MTU auto-flush + gap-detecting subscriber.
- **Gap Recovery**: Bounded ring of journaled MoldUDP64 messages backing a SoupBinTCP-over-TCP retransmission service.
- **Shared Memory IPC**: L2 market data distribution to co-located consumers via `shm_open`.

Kernel-Bypass Ingestion (DPDK / F-Stack)

Optional kernel-bypass order-entry path behind `#ifdef OB_HAVE_DPDK` (CMake `-DENABLE_DPDK=ON`): `DpdkGateway` runs F-Stack (DPDK userspace TCP over the AWS ENA PMD) on a dedicated **secondary ENI**, leaving the primary interface kernel-managed so SSH is never lost. A **single RX queue / single busy-poll lcore** preserves FIFO submit order (RSS splitting one connection across lcores would reorder and break price-time priority). F-Stack owns the NIC and runs FreeBSD's TCP stack internally; the gateway consumes its BSD-socket API (`ff_recv`), so ordered TCP payload flows straight into `OuchSession::feed()` and the engine submit path **unchanged** — DPDK replaces only how bytes arrive from the NIC, not how they are parsed. When `OB_HAVE_DPDK` is undefined (the default, and every non-Linux build) the kernel `TcpGateway` is the only ingestion path and the binary is byte-for-byte identical. **Implemented and locally verified** (macOS builds cleanly with the DPDK path `#ifdef` 'd out; gateway tests green); **AWS validation pending** via `scripts/dpdk_aws.sh`. With the kernel-TCP wire-to-wire baseline now measured (see below), the DPDK-vs-kernel-TCP comparison — **Session 2** — is the next step.

Wire-to-Wire Latency Measurement

A two-host OUCH-over-TCP harness — `tools/WireLatencySender` (instance A) and `tools/WireLatencyReceiver` (instance B) — measures NIC-to-NIC order-entry latency. On Linux it captures NIC **hardware** timestamps via `SO_TIMESTAMPING` (TX from the socket error queue, RX from the `recvmsg` cmsg), with a `CLOCK_MONOTONIC` **software fallback** on non-Linux / non-HW hosts (each CSV row records which source it used). One-way latency is estimated as **round-trip / 2**, which sidesteps clock synchronisation entirely — both timestamps are on the sender's own clock, so no PTP is needed for a first-pass number. The receiver also supports the F-Stack path (`--conf`) so the DPDK and kernel sessions emit directly comparable CSV, and a `--software-timestamps` flag forces both onto an identical CLOCK_MONOTONIC basis. **Implemented and baseline-validated on AWS**: a two-instance run (two `c6in.metal`, same AZ, kernel TCP, 10,000 OUCH orders, software timestamps on `CLOCK_MONOTONIC`) measured **P50 RTT 48.2 µs / P99 RTT 55.3 µs**, i.e. a **P50 one-way estimate of 24.1 µs** (round-trip / 2). The DPDK-vs-kernel-TCP comparison (Session 2) is the next step, via `scripts/wire_latency_aws.sh`.

---

4. Regulatory & Risk Management

Feature	Implementation
Self-Match Prevention (SMP)	Cancel-Taker strategy — prevents wash trading
Circuit Breakers	Configurable % price-band breach enters a short LULD-style **volatility auction** (orders accumulate without continuous matching until the reopening uncross) rather than a hard halt; emits a `breaker_trip` audit event
OTR Monitoring	Real-time Order-to-Trade ratio tracking per participant
Kill Switch	Instant cancellation of all orders per participant across all symbols
Pre-Trade Risk Limits	Max order size, notional value, and position limits per participant
Rate Limiting	Token-bucket throttling at ingress (configurable rate + burst); `RateLimiter::reconfigure()` live-updates default rate/burst and clears all participant buckets — called on SIGHUP
Queue Backpressure	Rejects orders when queue exceeds configurable threshold (default 80%)
LMM/DMM Privileges	`ParticipantRole` enum; ProRata 40% floor guarantee to LMM/DMM orders; remainder priority

---

5. Microstructure Research Infrastructure

A self-contained quantitative research layer that runs against the live matching engine, enabling empirical microstructure analysis and strategy development.

Component	Purpose
`SimulationDriver`	Multi-agent market simulator (NoiseTrader, MarketMaker, InformedTrader)
`MicrostructureMetrics` / `ResearchHarness`	Snapshot collection and research session management
`VPINCalculator`	Volume-Synchronized Probability of Informed Trading
`SpreadDecomposition`	Huang-Stoll and Glosten-Harris decomposition (adverse selection, inventory, order processing components)
`OrderFlowAnalytics`	Roll spread, queue imbalance signal, logistic fill-probability model

`ImpactModel`	Almgren-Chriss + square-root market impact laws
`ExecutionAnalytics` / `OptimalExecution`	Almgren-Chriss execution schedule, arrival-price and VWAP slippage
`CalibrationPipeline`	Nelder-Mead minimization of spread decomposition residuals
`BacktestEngine`	Historical tape replay against research models
`SignalGenerator`	Multi-factor composite signal (momentum, spread, VPIN, imbalance)
`PaperTrader`	Signal-driven IOC order submission with position and P&L tracking
`ResearchDashboard` / `ResearchSerializer`	Result visualization and persistence

BCS Latency-Arms-Race Study (`bcs_research/`)

A separate Python research layer that drives the verified C++ engine through a pybind11 bridge (`bcs_engine`) to test the Budish–Cramton–Shim theory of the HFT arms race experimentally. Because the matching core is TLA+-verified, emergent arms-race dynamics are attributable to agent incentives rather than to matcher error.

Component	Purpose
`agents/`	`BCSMarketMaker` (adverse-selection-aware, latency-disadvantaged), `HFTAgent` (snipes stale quotes), `NoiseTrader` (optional lognormal heavy-tailed sizes)
`simulation/scheduler.py`	`LatencyScheduler` — per-agent observation and submission latency
`metrics/`	Welfare decomposition (closed zero-sum identity), liquidity-gap detector, Kyle's lambda
`experiments/`	Exps 1–4: welfare transfer, latency sweep, HFT-count sweep, batch-auction remedy — plus `run_calibrated.py`
`calibration/`	Gap-safe moment extraction from live BTCUSDT-perp order flow and method-of-moments inversion with fill-rate correction
`analysis/`	Figure generation and the Hawkes endogeneity diagnostic
`paper/bcs_paper_draft.md`	The write-up (47 pp.)

Each experiment is reported twice: at a pre-registered operating point chosen for mechanism clarity, and re-run at environment parameters fitted to 27 days of Binance BTCUSDT-perpetual order flow (37.0M trades, 4.16M book snapshots). Every qualitative finding survives calibration. The calibration also supplies the study's only external check on magnitude: normalised by traded notional, HFT rent is 0.125 bp at the operating point and 4.42 bp calibrated, bracketing the ≈0.4 bp latency-arbitrage tax Aquilina, Budish and O'Neill (2022) measure on real exchange message data.

Two results are stated as conditional rather than calibrated. The calibrated rent is an upper bound, because a single latency-disadvantaged maker has no competitor to replenish a cleared quote and so absorbs the entire race. And the *incidence* of the transfer turns on a snipe-to-quote size ratio no public feed identifies:

at the operating point's ratio the maker bears the loss, while at a ratio of 0.024 the maker's PnL delta is positive for small HFT counts and noise traders bear it instead.

---

6. Reliability & High Availability

CRC-32 Journaling

- Write-ahead log with atomic CRC-32 integrity on every entry.
- Crash recovery: replay stops at first corrupted/truncated record.
- Atomic checkpoint: snapshots active book state, rewrites journal.
- Deterministic replay via virtual clock (`setExpiryClock(ClockFn)`).
- Auto-rotation: `OB_JOURNAL_MAX_SIZE_MB` env var triggers `engine.checkpoint()` from the main loop when `Journal::needsCheckpoint()` fires.

Cross-Host Log Replication (wired end-to-end)

- `ReplicationCoordinator` — TCP log-shipping from primary to backup, instantiated in `src/main.cpp` driven by `OB_NODE_ROLE` / `OB_PRIMARY_HOST` / `OB_JOURNAL_PATH` env vars.
- `Journal::onCommit` hook ships each fsync-durable batch to the backup; backup's `applyReplicatedEntry` writes to its own journal so the replica is durable across its own restarts.
- `HeartbeatMonitor` — configurable failure detection timeout.
- `LeaderLease` + `LeaseGrant` propagation — primary broadcasts lease state every heartbeat tick; backup's `tryAcquire` is fenced on local lease expiry, not just heartbeat miss. Prevents split brain under partial network failure (verified by 19-scenario chaos suite: partition, asymmetric partition, 30% packet loss, +30s clock skew — 0 split brain in all).
- Transport auto-reconnect — `receiveLoop` re-runs `connectTo()` against saved host/port after socket loss; ~3 ms recovery in chaos tests. Reconnect uses exponential backoff: 500ms → 30s cap, resets to base on successful connect.
- Snapshot catchup on join — primary streams every resting order via `MatchingEngine::streamSnapshot` when a backup connects; idempotent on receiver.
- Automatic backup promotion on combined heartbeat-miss + lease-expiry; `setReplayModeAllBooks(false)` flips backup into a live primary.

---

7. Observability & Operations

Observability Features

- **Structured Logging**: Pluggable `StructuredSink` API (Null, JSON Stderr, Capturing). 7 typed `obSink().log()` events wired in `OrderBook.cpp` (accepted/cancelled/rejected/fill/breaker/state); backends hot-swappable via typed helpers in `StructuredLog.h` .
- **Prometheus Metrics**: `MetricsRegistry` with `/prometheus` text-exposition endpoint. Tracks order flow, queue depth, latencies, and journal health.
- **Webhook Alerting**: `AlertDispatcher` with targets for Slack, PagerDuty, or HTTP webhooks.

Admin HTTP Server (Port 8080)

Endpoint	Purpose
`GET /health`	K8s liveness probe

`GET /readyz`	K8s readiness probe — HTTP 503 until warmup, then 200; auth-exempt; separate from liveness `/health`
`GET /metrics`	Internal counters (JSON)
`GET /prometheus`	Prometheus text exposition
`GET /book?symbolId=0`	L2 order book snapshot
`GET /otr?participantId=1`	Order-to-trade ratio

---

8. Performance Benchmarks

`HonestBenchmark` feeds one deterministic order flow (50K orders, seed=42) through three cumulative paths. The **x86 figures are the authoritative, reproducible numbers**, validated on AWS bare metal; Apple Silicon dev-machine numbers follow as a reference. P50 is the stable per-operation figure; throughput is wall-clock and load-sensitive. (Per-order/per-fill structured logging and event dispatch are sink-/listener-gated, so the hot path stays allocation- and vtable-free when nothing is attached.)

Validated x86 — AWS c6in.metal (authoritative, standard Release build)

Dual-socket Intel Xeon Platinum 8375C @ 2.90 GHz, hyperthreading disabled (`nosmt`), Ubuntu 26.04, Clang C++20 `-O3 -march=native` , `numactl --cpunodebind=0 --membind=0` . 5 stable runs, post-optimization commit `d2e688c` .

Path	What's Included	P50	P90	P99	Throughput
Core matching	OrderBook + STP + WashTrade + LULD	**261 ns**	620 ns	1,010 ns	2.80M ops/s
Engine wrapper	+ sequence alloc, rate limiter	269 ns	620 ns	1,001 ns	2.74M ops/s
Full-stack journal	+ GroupCommit (batch=64, async io_uring ack on EBS)	615 ns	1,048 ns	3,568 ns	1.28M ops/s

PGO: Clang IR-based profile-guided optimization (profiled on the seed=42 HonestBenchmark workload) takes Path A core matching to **P50 237 ns / P99 910 ns / 3.10M ops/s** — the headline figure. The table above is the standard (non-PGO) Release build.

`perf` (Path A, 50K orders seed=42): IPC 1.34 · 17.8 branch-misses/order · 152 L1-dcache-misses/order · 12,638 instructions/order.

Apple Silicon (M-series dev machine, reference)

Path	P50	Note
Core matching	~125 ns	~42 ns clock granularity quantizes per-path P50s, so Path A/B can read equal or invert run-to-run
Engine wrapper	~125 ns	

Full-stack journal	~1,400 ns	macOS/APFS `fdatasync` artifact — **not structural** (the same path is 615 ns on Linux x86, async io_uring ack)

The ~2.2× ARM-vs-x86 gap on core matching is microarchitectural (wider out-of-order window + stronger branch prediction on pointer-chasing code), **confirmed not** caused by build flags, field ordering, branch hints, or branchless selection.

Where the 261 ns goes (and what doesn't move it)

The four earlier micro-fixes (listener-dispatch guard, `shared_mutex`→plain `mutex`, OCO scratch-buffer reuse, rehash guard) produced **no measurable x86 latency change** — the P50 is **structurally bound**, not instruction-bound (17.8 branch-misses/order, 152 L1-dcache-misses/order). This cycle's branchless price-cross *did* move it, shaving 10 ns P50 / 62 ns P99 to reach 261 ns (see Optimization History in BENCHMARKS.md); next-order prefetch and the price-level arena allocator were both implemented and reverted as net-negative on this 100%-fill flow.

Cost	ns	Driver	Lever (estimated)
Pointer chasing (intrusive list)	80–100	152 L1-dcache misses/order	arena allocator (reverted — net-negative)
Branch mispredicts	60–80	17.8/order, data-dependent	branchless price-cross (shipped, −10 ns P50)
Irreducible work	50–60	price/qty math, STP, compliance	—
Spectre mitigation (eIBRS)	30–40	kernel-enforced on this instance	not disableable here

Confirmed **0 ns delta** on this workload: `-O2` vs `-O3`, `Order` field reordering, `[[likely]]` / `[[unlikely]]` hints, branchless `isBuy` book selection. Shipped this cycle: branchless price-cross (−10 ns P50 / −62 ns P99) and io_uring async journal ack (Path C P99 5,312→3,568 ns); next-order prefetch and the price-level arena allocator were both reverted as net-negative. Clang IR-based PGO then took Path A to 237 ns P50 (the headline figure). The journal P99 (~3.6 µs) is the `fdatasync`/EBS flush; NVMe/RAM-backed storage would be materially lower.

Binary Codec Microbenchmark

Codec	Message	ns/op	M ops/s
OUCH 4.2	EnterOrder encode (49B)	112.0	8.9
ITCH 5.0	AddOrder encode (36B)	34.1	29.4
SBE	**NewOrderV1 encode (32B)**	**1.0**	**1015**
SBE	**NewOrderV1 decode (32B)**	**0.5**	**2128**

---

9. Verification & Testing

Test Suite — 77 Executables, 426 CTest Targets

The testing infrastructure includes Unit, Functional, Integration, Chaos, Property, Shadow, and Benchmark testing categories across 77 test executables and 426 CTest targets. Key mechanisms:

- **Shadow Mode**: Dual-book divergence detection, validating FIFO compliance.
- **Fault Injection**: 10+ injection points (short-writes, pool exhaustion, EAGAIN injection) with zero-cost overhead in production.
- **Coverage-Guided Fuzzing**: libFuzzer harness for protocol parsing and order flow.
- **Sanitizers**: ASan, UBSan, and TSan checks integrated into CI/CD.

Formal Verification (TLA+)

12 TLA+ specifications, model-checked with TLC:

- **`MatchingEngine.tla`** : 1.26M distinct states verified, zero violations — now including the matching/cross layer (`Match` action) with `MatchingConservation` and `FIFOExecution` , alongside `NoNegativeQuantity` , `FIFO_Preservation` , `GTD_Expiry_Correctness` .
- **`Replication.tla`** : Realistic lease-propagation model (no god-mode `~primaryAlive` guard on `BackupPromote` ; promotion requires both heartbeat-miss AND local-lease-expiry). 1373 → 4192 states verified at `MaxEntries=6 → 10` , zero violations. A bug-injected variant (lease check stripped) reproduces split brain in 188 states, confirming the verification is genuine.
- **`MpscQueue.tla`** : Lock-free ring buffer linearizability.
- **`SnapshotLocked.tla`** : Mutex torn-snapshot prevention.

---

10. File Structure

```
include/              84 header files — core logic and networking
src/                  13 source files — thin compilation units
tests/                77 test files, 426 CTest targets
benchmarks/           9 benchmark binaries
fuzz/                 9 files — coverage-guided fuzzing harnesses
spec/                 TLA+ formal specifications (12 specs)
tools/                CLI tools and data utilities
config/               Example configuration files
docs/                 Runbooks, CapacityPlanning, ProductionReadiness
```

---

11. Remaining Work

The system is architecturally complete. The remaining items are AWS validation steps for capabilities already implemented and locally verified (nothing is hardware-blocked — the kernel-bypass path runs on commodity AWS ENA hardware):

- **x86 Bare Metal Benchmarks**: ✅ done — validated on AWS c6in.metal (dual Xeon 8375C, `nosmt` , NUMA-pinned); see §8. Core matching 261 ns P50, confirming the structural-bottleneck analysis (pointer-chasing L1 misses + data-dependent branch mispredicts + Spectre eIBRS). The prior four micro-optimizations moved x86 latency 0 ns; this cycle's branchless price-cross shaved 10 ns while next-order prefetch and the arena allocator were both reverted as net-negative — confirming the P50 is structurally bound.
- **Journal Async I/O (io_uring)**: ✅ done — validated on x86 Linux (AWS c6in.metal). The async ack (onCommit fires on the completion-reaper thread, not on submit) cut Path C P99 **32.8%** (5,312 → 3,568 ns) at the cost of +167 ns P50. io_uring is a generic Linux async-I/O interface and needs no

special NIC — only `liburing` on a modern kernel; the seam stays behind `#ifdef __linux__` with an `fdatasync` / `F_FULLFSYNC` fallback elsewhere.

- **Kernel Bypass Networking (DPDK / F-Stack)**: ⏳ implemented locally, pending AWS validation. `DpdkGateway` integrates F-Stack (DPDK userspace TCP over the AWS ENA PMD) on a secondary ENI behind `#ifdef OB_HAVE_DPDK`, feeding `OuchSession::feed()` unchanged (see §3). The integration is complete and builds locally with the path `#ifdef` 'd out; `scripts/dpdk_aws.sh` provisions the receiver (hugepages, DPDK + F-Stack install, vfio-pci bind, F-Stack config). It runs on **commodity AWS ENA** hardware — no Solarflare/Onload — needing only a secondary ENI, hugepages, and the DPDK/F-Stack toolchain; with the kernel-TCP wire-to-wire baseline now measured (see below), the two-instance c6in.metal run — **Session 2**, the DPDK-vs-kernel-TCP comparison — is the next step.
- **`Replication.tla` Verification**: ✅ done — see Verification section above. The original "not yet verified" footnote is obsolete.
- **BCS Study — Market-Maker Enrichment**: ⏳ open. The environment calibration (§5) is complete and the full experimental grid has been re-run at fitted parameters, but the model still has a single market maker. That is why the calibrated rent is reported as an upper bound rather than an estimate, and adding competing makers with heterogeneous latencies is the single change that would most improve the study's external validity. A second open item is the Hawkes diagnostic's real-data leg, which needs a depth-depletion gap definition for books that never empty.
- **Wire-to-Wire Latency Validation**: 🟡 partially complete — baseline measured, DPDK comparison pending. The `WireLatencySender` / `WireLatencyReceiver` harness (`SO_TIMESTAMPING` hardware timestamps with `CLOCK_MONOTONIC` software fallback, round-trip/2 one-way estimate — see §3) is validated on AWS: a two-instance run (two `c6in.metal`, same AZ, kernel TCP, 10,000 OUCH orders, software timestamps on `CLOCK_MONOTONIC`) measured **P50 RTT 48.2 µs / P99 RTT 55.3 µs**, i.e. a **P50 one-way estimate of 24.1 µs**. The remaining step is the DPDK-vs-kernel-TCP comparison (**Session 2**) via `scripts/wire_latency_aws.sh`.

Developed for professional quantitative trading systems. C++20 · ~46,500 LOC · 77 test executables · 426 CTest targets · 19 multi-container chaos scenarios · 1.26M TLA+ states verified on the matching-inclusive MatchingEngine.tla · 12 TLA+ specifications